4 agents in version 1 · 3 more later · 2 apps · 3 failure drills · 2 to 3 days

Scope · Flow · Tech roles · Agents · Topics · Messages · Redis and S3 · API · Risk levels · Failure drills · Build order · Done when

Scope

Version 1

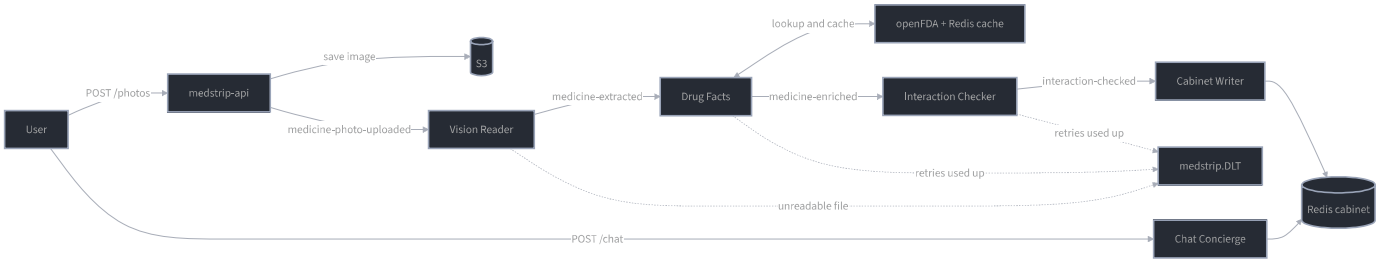
- Upload a strip photo and get a job id back right away.
- Vision Reader, Drug Facts, Interaction Checker, Chat Concierge.
- Cabinet of medicines per user, stored in Redis.
- Retry, dead letter topic, and a "retake photo" outcome.

Later

- Safety Reviewer (next, most interesting).
- Schedule Planner and Expiry Watcher.
- Login, real users, a front end beyond a simple page.

Safety rules. This is a learning project, not medical advice. Use your own photos or sample strips, never someone else's. No agent gives dosage advice or a diagnosis. Show "Not medical advice. Ask a pharmacist." in every chat answer and on every HIGH or UNKNOWN result.

Flow



Solid lines are version 1. Dashed lines are failure paths. When you add the Safety Reviewer, it sits between Interaction Checker and Cabinet Writer.

Job status the user sees

`UPLOADED` → `READING` → `LOOKING_UP` → `CHECKING` → `DONE` or `NEEDS_RETAKE` `FAILED`

What each technology does

TECHNOLOGY	JOB IN THIS PROJECT
S3 (LocalStack locally)	Stores uploaded photos. Key is `{userId}/{jobId}.jpg`. Delete after 7 days.
Kafka	Moves a job from one agent to the next. The upload returns at once and the agents work in the background.
Redis	Cabinet, job status, openFDA cache, duplicate-photo check, upload rate limit, chat memory.
Spring AI	The pipeline agents: vision reading, summarizing label text, risk rating. Structured output into Java records.
LangChain4j	The Chat Concierge: an AI service with tools and Redis-backed chat memory.
openFDA	Free drug label API. Fields used: `warnings`, `drug_interactions`. Example: `https://api.fda.gov/drug/label.json?search=openfda.brand_name:"ibuprofen"&limit=1`

Two Spring Boot apps: **medstrip-api** (upload, status, cabinet, chat) and

Two Spring Boot apps: **medstrip-api** (upload, status, cabinet, chat) and **medstrip-agents** (all Kafka consumers). Run Kafka, Kafka UI, Redis and LocalStack with Docker Compose. Use any vision-capable model, or a local one through Ollama.

Agents

Vision Reader — v1

LISTENS	`medicine-photo-uploaded`
DOES	Loads the image from S3 and asks the model for name, strength, expiry and a confidence score.
PRODUCES	`medicine-extracted`
EDGE CASES	Confidence under 0.7 sets `NEEDS_RETAKE` and stops. Not an image goes to the DLT.

Drug Facts — v1

LISTENS	`medicine-extracted`
DOES	Calls openFDA as a tool, summarizes warnings in 5 bullets or fewer, caches the result for 24 hours.
PRODUCES	`medicine-enriched`
EDGE CASES	3 second timeout, backoff retries. If openFDA stays down, passes on `source: "unavailable"`.

Interaction Checker — v1

LISTENS	`medicine-enriched`
DOES	Reads the user's cabinet, compares it with the label text, returns a risk level and reasons.
PRODUCES	`interaction-checked`
EDGE CASES	No label text means `UNKNOWN`, never `LOW`.

Chat Concierge — v1

RUNS IN	medstrip-api, using LangChain4j
TOOLS	`listCabinet`, `getDrugWarnings(name)`, `getInteractionResult(name)`
MEMORY	Last 10 messages per user, stored in Redis.
RULES	Answers only from tool results. Refuses dosage advice and diagnosis.

Safety Reviewer — next

LISTENS	`interaction-checked`

Schedule Planner — later

LISTENS	`medicine-approved`

DOES — Acts as a critic. Removes dosage or diagnosis wording and adds "ask a pharmacist" on HIGH and UNKNOWN.

PRODUCES — `medicine-approved` with `reviewNotes`

DOES — Builds a daily plan from the frequency the user enters and label hints such as "take with food". Saves it as `schedule:{userId}`.

Expiry Watcher

later

RUNS — Once a day on a scheduler.

DOES — Finds cabinet items expiring within 30 days.

PRODUCES — `expiry-alert`

Cabinet Writer

v1

TYPE — Plain code, not an agent.

LISTENS — `interaction-checked`, later `medicine-approved`

DOES — Saves the medicine to the cabinet and sets the job to `DONE`. Same name, strength and expiry is saved once.

Kafka topics

TOPIC	PRODUCER	CONSUMER	KEY
`medicine-photo-uploaded`	medstrip-api	Vision Reader	`userId`
`medicine-extracted`	Vision Reader	Drug Facts	`userId`
`medicine-enriched`	Drug Facts	Interaction Checker	`userId`
`interaction-checked`	Interaction Checker	Cabinet Writer, later Safety Reviewer	`userId`
`medicine-approved`	Safety Reviewer	Cabinet Writer, Schedule Planner	`userId`
`expiry-alert`	Expiry Watcher	Log or notify	`userId`
`medstrip.retry`	Any agent	Same agent after a delay	`userId`

medstrip.DLT	Any agent	You, by hand	original key

Use 3 partitions on each main topic. The key is `userId` so one user's messages stay in order within each step.

> Order is guaranteed per step, not across the whole pipeline. If one user uploads two photos within a few seconds, the second check can run before the first medicine reaches the cabinet. Accept this in version 1. It makes a good stretch drill later.

Message contracts

Every message carries `jobId` , `userId` and `schemaVersion` . Examples for the two messages you will debug most:

```
// medicine-extracted
{
  "jobId": "J-1042",
  "userId": "U1",
  "name": "Ibuprofen",
  "strength": "400 mg",
  "expiry": "2027-03",
  "confidence": 0.93,
  "schemaVersion": 1
}
```

```
// interaction-checked
{
  "jobId": "J-1042",
  "userId": "U1",
  "name": "Ibuprofen",
  "risk": "MEDIUM",
  "conflictsWith": ["Aspirin"],
  "reasons": ["Label advises caution with
  "source": "openFDA",
  "schemaVersion": 1
}
```

MESSAGE	EXTRA FIELDS
medicine-photo-uploaded	s3Key , sha256 , uploadedAt
medicine-enriched	name , warnings[] (max 5), interactionText , source = openFDA , cache or unavailable
medicine-approved	Same as interaction-checked plus reviewNotes
expiry-alert	name , expiry , daysLeft

Redis and S3 layout

KEY	TYPE	HOLDS	EXPIRES
`job:{jobId}`	hash	status, userId, error reason, last step done	7 days
`cabinet:{userId}`	hash	medicine id → JSON (name, strength, expiry, last risk)	none
`fda:{normalizedName}`	string	summarized label text and source	24 hours
`photo:{sha256}`	string	jobId of the first upload of this photo	1 hour
`rate:{userId}`	counter	uploads in the current minute (limit 5)	60 seconds
`chat:{userId}`	list	last 10 chat messages	24 hours

S3 bucket `medstrip-photos`, key `{userId}/{jobId}.jpg`, lifecycle rule to delete after 7 days. Point the AWS SDK at the LocalStack endpoint while developing.

REST API (medstrip-api)

No login. Send a synthetic user in the `X-User-Id` header.

ENDPOINT	BEHAVIOUR
`POST /photos`	Multipart image. Checks the rate limit (`429`), the duplicate hash (returns the existing jobId), then saves to S3, sets `UPLOADED`, publishes to Kafka, returns `202` with `jobId`.
`GET /jobs/{jobId}`	Status, plus the result and reasons when finished.
`GET /cabinet`	All medicines for the user with expiry and last risk.

`DELETE /cabinet/{medicineId}`	Remove a medicine.
`POST /chat`	Body `{"message": "..."}`. Returns the Concierge answer.

Risk levels

LEVEL	MEANING	SHOWN TO THE USER AS
LOW	The label text mentions no interaction with anything in the cabinet.	"No interaction found in the label. This is not a guarantee."
MEDIUM	The label advises caution, or two items share an ingredient class.	The reason, plus "check with a pharmacist".
HIGH	The label warns against combining it with an item in the cabinet.	The reason, plus "ask a pharmacist before taking both".
UNKNOWN	No label text was found, or the lookup failed.	"Could not check. Ask a pharmacist." Never shown as safe.

Failure drills

Build the success path first. For each drill, watch it break before you add the fix.

DRILL	WHAT BREAKS	REQUIRED BEHAVIOUR	HOW TO TRIGGER
1 Blurry photo	Model returns a low confidence guess. Retrying will not help.	Treat it as a result, not an error. Status `NEEDS_RETAKE`, nothing sent to the DLT.	Upload a dark or blurred photo
2 Bad model output	Model returns text that does not parse into the record.	Retry twice, then DLT and status `FAILED`. Other jobs keep flowing.	Make the prompt ask for free text, or mock the model

3 openFDA down	Calls hang, the consumer falls behind, lag grows.	3 second timeout, backoff, serve from cache if present. If not, pass on `unavailable` and rate the result `UNKNOWN`.	Block `api.fda.gov` or point the client at a dead port
4 Duplicate photo	Same photo uploaded twice, or a message replayed, creates two cabinet entries.	Hash check returns the first jobId. Cabinet write is idempotent on name, strength and expiry.	Upload the same file twice, or replay the topic from the start
5 Worker crash	Agent dies after doing its work but before the next step.	Commit the offset only after the next event is sent. Each step checks `job:{jobId}` so a replay does nothing twice.	Kill medstrip-agents mid-job, then restart
6 S3 object missing	Event arrives but the image cannot be loaded.	Retry with backoff in case of a short delay. Then DLT with the reason and status `FAILED`.	Delete the object before the Vision Reader runs

Build order

DAY 1	
1	Docker Compose with Kafka, Kafka UI, Redis and LocalStack. Create the topics and the bucket.
2	**medstrip-api:** `POST /photos` saves to S3, writes the job to Redis, publishes the event.
3	**Vision Reader** with Spring AI structured output. Test with three real strip photos.
4	**Drug Facts** with the openFDA call and the Redis cache.
DAY 2	
5	**Interaction Checker** and **Cabinet Writer**. Upload two medicines and see the risk result.
6	**Chat Concierge** with LangChain4j: tools first, then Redis chat memory.
7	Failure drills 1 to 3

7	Failure drills 1 to 3.
DAY 3, IF YOU HAVE TIME	
8	Failure drills 4 to 6.
9	**Safety Reviewer** between Interaction Checker and Cabinet Writer.

Done when

0 of 10 checked

- [ ] A clear photo goes from upload to `DONE` and the medicine appears in `GET /cabinet`.
- [ ] Upload returns in under a second, before any agent has finished.
- [ ] A blurry photo ends as `NEEDS_RETAKE` and nothing lands in the DLT.
- [ ] The second medicine is checked against the first and the risk level is stored.
- [ ] A second lookup of the same drug is served from Redis, not openFDA.
- [ ] With openFDA blocked, the result is `UNKNOWN` and never `LOW`.
- [ ] Uploading the same photo twice returns the same jobId and one cabinet entry.
- [ ] Killing the agents app mid-job and restarting still ends in `DONE` with one cabinet entry